Notes

Web Development

Unit 1

1. Introduction to React & Project Setup

What is React?

React is an open-source JavaScript library for building user interfaces, developed and maintained by Facebook. It's used for creating interactive, stateful, and reusable UI components that update efficiently when data changes.

Key Features of React:

1. **Component-Based Architecture**: React applications are built using reusable components that manage their own state and render UI.
2. **Virtual DOM**: React creates a virtual representation of the DOM in memory, which allows it to calculate the most efficient way to update the browser's DOM.
3. **Declarative Syntax**: You describe what your UI should look like for each state, and React takes care of updating the DOM to match.
4. **Unidirectional Data Flow**: Data in React flows in a single direction, making it easier to understand how your application works.
5. **JSX Syntax**: A syntax extension that allows you to write HTML-like code in your JavaScript.
6. **Rich Ecosystem**: Includes tools like React Router for navigation, Redux for state management, and many other supporting libraries.

Why Use React?

- **Performance**: Efficient updates with Virtual DOM
- **Reusability**: Component-based structure promotes code reuse
- **Community**: Large ecosystem and active community support
- **Flexibility**: Can be used for web, mobile (React Native), and even VR applications
- **Employability**: One of the most in-demand skills for frontend developers

Project Setup

There are several ways to set up a React project. Here are the most common approaches:

1. Using Create React App (CRA) (Deprecated)

Create React App is the official way to create single-page React applications with no build configuration.

Steps to set up:

1. **Install Node.js**: Ensure you have Node.js (v14 or later) installed on your machine.
2. **Create the app**:

   npx create-react-app my-app
3. **Navigate to the project directory**:

   cd my-app
4. **Start the development server**:

   npm start

This will:

- Set up a new React project in the my-app directory
- Install all required dependencies
- Start a development server (usually at http://localhost:3000)
- Open your default browser to view the app

2. Using Vite

Vite is a newer, faster build tool that's gaining popularity in the React community.

Steps to set up with Vite:

1. **Create the app**:

   npm create vite@latest my-react-app --template react
2. **Navigate to the project directory**:

   cd my-react-app
3. **Install dependencies**:

   npm install
4. **Start the development server**:

   npm run dev

3. Manual Setup (Advanced)

For more control, you can set up React manually with Webpack and Babel:

1. Initialize a new project:

   npm init -y
2. Install React and React DOM:

npm install react react-dom

3. Install development dependencies:

   npm install --save-dev webpack webpack-cli webpack-dev-server babel-loader @babel/core @babel/preset-env @babel/preset-react html-webpack-plugin css-loader style-loader

4. **Create configuration files for Webpack and Babel**

Project Structure (CRA Example)

A typical React project created with CRA has this structure:

```
my-app/
├── node_modules/      # All project dependencies
├── public/            # Static files
│   ├── index.html     # Main HTML template
│   └── ...            # Other static assets
├── src/               # Application source code
│   ├── App.css        # Main styles
│   ├── App.js         # Main App component
│   ├── App.test.js    # Tests for App component
│   ├── index.css      # Global styles
│   ├── index.js       # Application entry point
│   └── ...            # Other components
├── package.json       # Project configuration and dependencies
├── package-lock.json  # Exact dependency versions
└── README.md          # Project documentation
```

Understanding the Key Files

1. **public/index.html**: The single HTML page that serves as the entry point.
2. **src/index.js**: The JavaScript entry point that renders the root React component.
3. **src/App.js**: The main React component that serves as the root of your component tree.

Running and Building

- **Development server**: npm start (runs on http://localhost:3000)

- **Production build**: npm run build (creates optimized files in the build folder)
- **Running tests**: npm test
- **Ejecting (CRA only)**: npm run eject (removes the single build dependency and copies all configuration files to your project)

Next Steps After Setup

1. **Create components**: Start building your application by creating React components
2. **Add routing**: Install React Router for navigation between views
3. **State management**: Consider adding Redux or Context API for state management
4. **Styling**: Choose a CSS approach (CSS Modules, styled-components, Tailwind, etc.)
5. **API integration**: Connect your app to backend services

React provides a powerful foundation for building modern web applications, and proper setup is the first step toward creating maintainable and scalable projects.

2. JSX, JSX Expressions & Rendering Basics

What is JSX?

JSX (JavaScript XML) is a syntax extension for JavaScript that allows you to write HTML-like code in your JavaScript files. It's a fundamental part of React development that makes writing and adding markup in React more intuitive.

Key Characteristics of JSX:

1. **Not a string or HTML**: JSX looks like HTML but is actually syntactic sugar for JavaScript function calls.
2. **Compiled to JavaScript**: JSX gets transformed into regular JavaScript by tools like Babel.
3. **Not required for React**: You can write React without JSX, but it's much more convenient with JSX.

Basic JSX Example:

const element = <h1>Hello, world!</h1>;

This gets compiled to:

const element = React.createElement('h1', null, 'Hello, world!');

JSX Expressions

JSX allows you to embed JavaScript expressions within the markup using curly braces {}.

Types of JSX Expressions:

1. **Embedding Variables**:

```
const name = 'John Doe';

const element = <h1>Hello, {name}</h1>;
```

2. **Embedding Function Calls**:

```
function formatName(user) {
  return user.firstName + ' ' + user.lastName;
}

const element = <h1>Hello, {formatName(user)}!</h1>;
```

3. **Embedding Expressions**:

```
const element = <h1>2 + 2 = {2 + 2}</h1>;
```

4. **Attribute Expressions**:

```
const element = <img src={user.avatarUrl} />;
```

5. **Boolean, Null, and Undefined**:
   These values don't render anything:

```
<div>{undefined}</div>

<div>{null}</div>

<div>{false}</div>

<div>{true}</div>
```

JSX Rendering Basics

Rendering JSX to the DOM

To render JSX elements to the DOM, you use ReactDOM.render():

```
const element = <h1>Hello, world</h1>;

ReactDOM.render(element, document.getElementById('root'));
```

Key Rendering Concepts:

1. **Single Root Element**: JSX must have a single root element. You can use fragments (<>...</>) to group elements without adding extra nodes:

```
const element = (
  <>
    <h1>Hello</h1>
    <h2>World</h2>
  </>
```

```
);
```

2. **Self-Closing Tags**: In JSX, all tags must be closed, even those that don't require closing in HTML:

   ```
   <img src="..." />

   <input type="text" />
   ```

3. **className instead of class**: Since class is a reserved word in JavaScript, JSX uses className:

   ```
   <div className="container"></div>
   ```

4. **htmlFor instead of for**: Similarly, for becomes htmlFor:

   ```
   <label htmlFor="name">Name</label>
   ```

5. **Inline Styles**: JSX uses objects for inline styles with camelCase property names:

   ```
   const divStyle = {
     color: 'blue',
     backgroundColor: 'lightgray'
   };

   <div style={divStyle}>Hello World</div>
   ```

6. **Comments in JSX**: Comments are written as JavaScript expressions:

   ```
   <div>
     {/* This is a comment */}
     <h1>Hello</h1>
   </div>
   ```

Conditional Rendering in JSX

JSX supports various patterns for conditional rendering:

1. **Ternary Operator**:

   ```
   <div>
     {isLoggedIn ? <LogoutButton /> : <LoginButton />}
   </div>
   ```

2. **Logical && Operator**:

   ```
   <div>
     {unreadMessages.length > 0 &&
   ```

```
    <h2>You have {unreadMessages.length} unread messages.</h2>
  }
</div>
```

3. **Immediately Invoked Function Expression (IIFE)**:

```
<div>
 {(() => {
   if (conditionOne) return <ComponentA />;
   if (conditionTwo) return <ComponentB />;
   return <ComponentC />;
 })()}
</div>
```

List Rendering in JSX

To render lists in JSX, you typically use the map() function:

```
const numbers = [1, 2, 3, 4, 5];
const listItems = numbers.map((number) =>
  <li key={number.toString()}>
    {number}
  </li>
);
ReactDOM.render(
  <ul>{listItems}</ul>,
  document.getElementById('root')
);
```

Note the key prop which helps React identify which items have changed.

JSX Under the Hood

When Babel processes JSX, it transforms it into React.createElement() calls:

```
<div className="container" id="app">
  <h1>Hello {name}</h1>
</div>
```

Gets compiled to:

```
javascript
React.createElement(
  'div',
  { className: 'container', id: 'app' },
  React.createElement('h1', null, 'Hello ', name)
);
```

This is why you need to import React even when you're only using JSX.

JSX vs HTML Differences

1. **Attributes are camelCase**: tabindex becomes tabIndex
2. **Some attribute names are different**: class → className, for → htmlFor
3. **Style attribute takes an object**: Not a string
4. **Self-closing tags are required**: <img /> instead of <img>
5. **Event handlers are camelCase**: onclick becomes onClick

JSX provides a more intuitive way to work with UI components in React, combining the power of JavaScript with the familiarity of HTML-like syntax.

3. Rendering Lists & Conditional Rendering

Rendering Lists in React

Rendering lists is a common task in React applications when you need to display collections of data. React provides powerful ways to render lists efficiently.

Basic List Rendering

The most common way to render lists is by using the map() function:

```
const numbers = [1, 2, 3, 4, 5];
const listItems = numbers.map((number) =>
  <li key={number.toString()}>
    {number}
  </li>
);
return (
  <ul>{listItems}</ul>
```

);

Key Concepts for List Rendering

1. **The** key **Prop**:
    - Keys help React identify which items have changed, are added, or are removed
    - Should be unique among siblings (but don't need to be globally unique)
    - Best to use stable IDs from your data if available
    - Avoid using array indices as keys if the list can reorder

```
const todoItems = todos.map((todo) =>
  <li key={todo.id}>
    {todo.text}
  </li>
);
```

2. **Inline Map in JSX**:
   You can embed the map directly in your JSX:

```
return (
  <ul>
    {numbers.map((number) =>
      <ListItem key={number.toString()} value={number} />
    )}
  </ul>
);
```

3. **Filtering Lists**:
   Combine filter() with map() to render filtered lists:

```
const visibleTodos = todos.filter(todo => !todo.completed);
const todoList = visibleTodos.map(todo => (
  <li key={todo.id}>
    {todo.text}
  </li>
));
```

4. **Handling Empty Lists**:

```
return (
 <div>
  {items.length > 0 ? (
   <ul>
    {items.map(item => (
     <li key={item.id}>{item.name}</li>
    ))}
   </ul>
  ) : (
   <p>No items found</p>
  )}
 </div>
);
```

Conditional Rendering in React

Conditional rendering allows you to show or hide elements based on certain conditions, similar to how if statements work in JavaScript.

Common Conditional Rendering Patterns

1. **If/Else Statements**:

```
function Greeting({ isLoggedIn }) {
 if (isLoggedIn) {
  return <UserGreeting />;
 }
 return <GuestGreeting />;
}
```

2. **Ternary Operator**:
   Great for simple conditions within JSX:

```
return (
 <div>
  {isLoggedIn ? (
```

```
      <LogoutButton onClick={handleLogout} />
    ) : (
      <LoginButton onClick={handleLogin} />
    )}
  </div>
);
```

3. **Logical && Operator**:
   Useful when you want to render something or nothing:

```
return (
  <div>
    {unreadMessages.length > 0 && (
      <h2>
        You have {unreadMessages.length} unread messages.
      </h2>
    )}
  </div>
);
```

4. **Immediately Invoked Function Expression (IIFE)**:
   For more complex logic within JSX:

```
return (
  <div>
    {(() => {
      if (error) return <ErrorComponent />;
      if (loading) return <Loader />;
      return <DataComponent data={data} />;
    })()}
  </div>
);
```

5. **Component Variables**:
   Store elements in variables before returning JSX:

```
function Mailbox({ unreadMessages }) {
  const messageCount = unreadMessages.length;
  let message;
    if (messageCount > 0) {
    message = <h2>You have {messageCount} unread messages.</h2>;
  } else {
    message = <h2>You have no unread messages.</h2>;
  }
    return <div>{message}</div>;
}
```

6. **Preventing Component Rendering**:
   Return null to hide a component:

```
function WarningBanner({ warn }) {
  if (!warn) {
    return null;
  }

  return <div className="warning">Warning!</div>;
}
```

Advanced Conditional Rendering Techniques

1. **Switch Statements**:

```
function Notification({ status }) {
  switch (status) {
    case 'success':
      return <SuccessNotification />;
    case 'warning':
      return <WarningNotification />;
    case 'error':
      return <ErrorNotification />;
    default:
```

```
    return <InfoNotification />;
  }
}
```

2. **Enum Objects**:
   Clean pattern for multiple conditions:

```
const NOTIFICATION_TYPES = {
  info: <Info />,
  warning: <Warning />,
  error: <Error />,
};
function Notification({ status }) {
  return <div>{NOTIFICATION_TYPES[status]}</div>;
}
```

3. **Higher-Order Components (HOCs) for Conditional Rendering**:

```
function withCondition(Component, condition) {
  return function ConditionalComponent(props) {
    if (condition(props)) {
      return <Component {...props} />;
    }
    return null;
  };
}
const AdminButton = withCondition(Button, props => props.user.isAdmin);
```

4. **Render Props Pattern**:

```
<ConditionalRender
  condition={isLoggedIn}
  renderTrue={() => <LogoutButton />}
  renderFalse={() => <LoginButton />}
/>
```

Performance Considerations

1. **Keys in Lists**:
    - Always use proper keys to help React's reconciliation algorithm
    - Avoid index as key when list order might change
2. **Conditional Rendering Optimization**:
    - For frequently toggled elements, consider CSS display property instead of unmounting
    - Use React.memo for components with expensive renders
3. **Inline Functions in Render**:
    - Avoid creating new functions in render when passing to child components
    - Use useCallback for stable function references

Practical Examples

List with Filter and Sort:

```
function ProductList({ products, filterText, inStockOnly }) {
 const filteredProducts = products
  .filter(product =>
   product.name.includes(filterText) &&
   (inStockOnly ? product.stocked : true)
  )
  .sort((a, b) => a.price - b.price);
 return (
  <ul>
   {filteredProducts.map(product => (
    <ProductItem key={product.id} product={product} />
   ))}
  </ul>
 );
}
```

Complex Conditional Rendering:

```
function AuthWidget({ user, isLoading, error }) {
 if (isLoading) return <Spinner />;
```

```
if (error) return (
  <div>
    <Error message={error.message} />
    <button onClick={retryAuth}>Retry</button>
  </div>
);
  return user ? (
  <div>
    <WelcomeMessage user={user} />
    <Dashboard />
  </div>
) : (
  <LoginForm />
);
}
```

These techniques form the foundation for building dynamic UIs in React, allowing you to create applications that respond to data changes and user interactions effectively.

4. Functional Components & Props

1. Functional Components in React

Functional components are JavaScript functions that return React elements (JSX). They are simpler and more concise than class components and are now the preferred way to write components in React (especially with Hooks).

Characteristics of Functional Components:

- **Pure JavaScript Functions**: They take props as input and return JSX.
- **No** this **Keyword**: Unlike class components, they don't use this.
- **Stateless (before Hooks)**: Initially, they were stateless, but now with Hooks, they can manage state and lifecycle.
- **Easier to Read & Test**: Due to their simplicity.

Basic Syntax:

```
function Welcome() {
  return <h1>Hello, World!</h1>;
}
```

or using **Arrow Functions**:

```
const Welcome = () => <h1>Hello, World!</h1>;
```

Using Functional Components:

```
import React from 'react';
function Greeting() {
  return <h1>Welcome to React!</h1>;
}
export default Greeting;
```

Then render it in another component:

```
import Greeting from './Greeting';
function App() {
  return (
    <div>
      <Greeting />
    </div>
  );
}
```

2. Props in Functional Components

Props (short for "properties") allow data to be passed from a parent component to a child component. They are **read-only** (immutable).

Passing Props to a Component

```
function App() {
  return (
    <div>
      <Greeting name="John" age={25} />
    </div>
  );
```

```
}
```

Here, name and age are props passed to Greeting.

Accessing Props in a Functional Component

```
function Greeting(props) {
  return (
    <h1>
      Hello, {props.name}! You are {props.age} years old.
    </h1>
  );
}
```

Alternative (Destructuring Props):

```
function Greeting({ name, age }) {
  return (
    <h1>
      Hello, {name}! You are {age} years old.
    </h1>
  );
}
```

Default Props

If a prop is not passed, you can define default values:

```
function Greeting({ name = "Guest", age = 18 }) {
  return (
    <h1>
      Hello, {name}! You are {age} years old.
    </h1>
  );
}
```

or using defaultProps (less common in modern React):

```
Greeting.defaultProps = {
  name: "Guest",
  age: 18,
};
```

Passing Functions as Props

```
function App() {
  const handleClick = () => alert("Button clicked!");
  return (
    <div>
      <Button onClick={handleClick} label="Click Me" />
    </div>
  );
}
function Button({ onClick, label }) {
  return <button onClick={onClick}>{label}</button>;
}
```

Children Prop (Special Prop)

The children prop allows passing components/elements inside another component:

```
function Card({ children }) {
  return <div className="card">{children}</div>;
}
function App() {
  return (
    <Card>
      <h1>Title</h1>
      <p>This is a card content.</p>
    </Card>
  );
}
```

3. Key Differences Between Props and State

Feature	Props	State
Mutability	Immutable (read-only)	Mutable (can change)
Ownership	Passed from parent	Managed within component
Purpose	Used for configuration	Used for dynamic data

4. Best Practices with Props

1. **Use Destructuring** for cleaner code:

```
function User({ name, email, role }) {
  return (
    <div>
      <p>{name}</p>
      <p>{email}</p>
      <p>{role}</p>
    </div>
  );
}
```

2. **Avoid Mutating Props** (Props should be read-only).
3. **Use PropTypes or TypeScript** for type checking:

```
import PropTypes from 'prop-types';
User.propTypes = {
  name: PropTypes.string.isRequired,
  email: PropTypes.string,
  role: PropTypes.oneOf(['admin', 'user']),
};
```

4. **Keep Components Small** and pass only necessary props.

5. Example: Complete Functional Component with Props

```
import React from 'react';
import PropTypes from 'prop-types';
function UserProfile({ name, age, isAdmin, onLogout }) {
 return (
  <div className="profile">
   <h2>Welcome, {name}!</h2>
   <p>Age: {age}</p>
   {isAdmin && <p>Admin Access</p>}
   <button onClick={onLogout}>Logout</button>
  </div>
 );
}
UserProfile.propTypes = {
 name: PropTypes.string.isRequired,
 age: PropTypes.number,
 isAdmin: PropTypes.bool,
 onLogout: PropTypes.func.isRequired,
};
UserProfile.defaultProps = {
 age: 18,
 isAdmin: false,
};
export default UserProfile;
```

Usage:

```
function App() {
 const handleLogout = () => console.log("Logged out!");
 return (
  <UserProfile
```

```
      name="Alice"
      age={25}
      isAdmin
      onLogout={handleLogout}
    />
  );
}
```

Summary

- **Functional Components** are simple JavaScript functions that return JSX.
- **Props** allow passing data from parent to child components.
- **Props are read-only** and should not be modified inside the component.
- **Destructuring props** makes code cleaner.
- **Default props** and **PropTypes** improve code reliability.
- **Children prop** allows nesting components.

Functional components + Props + Hooks (like useState, useEffect) form the modern way of building React applications.

5. Styling in React & Using Bootstrap

1. Styling in React

React provides multiple ways to style components. Here are the most common approaches:

1. Inline Styles

- Uses JavaScript objects (not CSS strings).
- Properties are camelCased (backgroundColor instead of background-color).
- Best for dynamic styles.

Example:

```
function Button() {
  const buttonStyle = {
    backgroundColor: 'blue',
    color: 'white',
    padding: '10px 20px',
    borderRadius: '5px',
```

```
    border: 'none',
    cursor: 'pointer',
  };
  return <button style={buttonStyle}>Click Me</button>;
}
```

Dynamic Styles Example:

```
function DynamicButton({ isActive }) {
  const buttonStyle = {
    backgroundColor: isActive ? 'green' : 'gray',
    color: 'white',
    padding: '10px 20px',
  };
  return <button style={buttonStyle}>{isActive ? 'Active' : 'Inactive'}</button>;
}
```

2. CSS Stylesheets

- Traditional CSS files (.css).
- Import directly into components.

Example:

```
/* Button.css */
.btn {
  background-color: blue;
  color: white;
  padding: 10px 20px;
  border-radius: 5px;
  border: none;
  cursor: pointer;
}
.btn:hover {
  opacity: 0.8;
```

```
}
```

Usage in React:

```
import './Button.css';
function Button() {
  return <button className="btn">Click Me</button>;
}
```

3. CSS Modules

- Avoids global scope issues by generating unique class names.
- File naming: [name].module.css.

Example:

```
/* Button.module.css */
.primary {
  background-color: blue;
  color: white;
}
```

Usage:

```
import styles from './Button.module.css';
function Button() {
  return <button className={styles.primary}>Click Me</button>;
}
```

4. Styled Components (CSS-in-JS)

- Uses tagged template literals (styled-components, emotion).
- Dynamic styling with props.

Example:

```
npm install styled-components
import styled from 'styled-components';
const StyledButton = styled.button`
  background-color: ${props => props.primary ? 'blue' : 'gray'};
  color: white;
```

```
  padding: 10px 20px;
  border-radius: 5px;
  border: none;
  cursor: pointer;
  &:hover {
    opacity: 0.8;
  }
`;
function Button() {
  return (
    <>
      <StyledButton>Normal</StyledButton>
      <StyledButton primary>Primary</StyledButton>
    </>
  );
}
```

5. Tailwind CSS (Utility-First CSS)

- Uses utility classes (bg-blue-500, p-4).
- No need to write custom CSS.

Example:

```
function Button() {
  return (
    <button className="bg-blue-500 text-white px-4 py-2 rounded hover:bg-blue-600">
      Click Me
    </button>
  );
}
```

2. Using Bootstrap in React

Bootstrap is a popular CSS framework. There are two ways to use it in React:

1. Using Bootstrap CDN (Traditional Way)

- Add Bootstrap CSS and JS via <link> in public/index.html.
- Not recommended for React (global scope issues).

Example:

```
<!-- public/index.html -->
<link
  href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.0/dist/css/bootstrap.min.css"
  rel="stylesheet"
/>
<script
src="https://cdn.jsdelivr.net/npm/bootstrap@5.3.0/dist/js/bootstrap.bundle.min.js"
></script>
```

Usage:

```
function Alert() {
  return (
    <div className="alert alert-primary" role="alert">
      A simple Bootstrap alert!
    </div>
  );
}
```

2. Using React-Bootstrap (Recommended)

- Official Bootstrap components rebuilt for React.
- Better integration with React.

Installation:

```
npm install react-bootstrap bootstrap
```

Example:

```
import 'bootstrap/dist/css/bootstrap.min.css';
import { Button, Alert, Card } from 'react-bootstrap';
function App() {
  return (
```

```
    <div className="container mt-4">
      <Alert variant="success">Success Alert!</Alert>
      <Button variant="primary">Click Me</Button>
          <Card style={{ width: '18rem' }}>
        <Card.Body>
          <Card.Title>React + Bootstrap</Card.Title>
          <Card.Text>Better integration with React.</Card.Text>
        </Card.Body>
      </Card>
    </div>
  );
}
```

3. Using Bootstrap with CSS Modules

- Avoids global class conflicts.

Example:

```
import styles from './App.module.css';
import 'bootstrap/dist/css/bootstrap.min.css';
function App() {
  return (
    <div className={`container ${styles.customContainer}`}>
      <button className="btn btn-primary">Bootstrap Button</button>
    </div>
  );
}
```

Comparison of Styling Methods

Method	Pros	Cons
Inline Styles	Dynamic, no CSS files	Hard to maintain, no pseudo-classes

Method	Pros	Cons
CSS Stylesheets	Traditional, easy	Global scope issues
CSS Modules	Scoped styles	Slightly more setup
Styled Components	Dynamic, scoped	Extra dependency
Bootstrap CDN	Quick setup	Global scope, not React-friendly
React-Bootstrap	Best for Bootstrap	Larger bundle size

Best Practices

1. **Use CSS Modules or Styled Components** for scoped styles.
2. **Avoid inline styles** for complex UIs.
3. **Prefer React-Bootstrap** over plain Bootstrap in React.
4. **Use Tailwind CSS** if you prefer utility-first styling.
5. **Keep styles modular** (component-based styling).

Final Example: Styled React App with Bootstrap

```
import { useState } from 'react';
import { Button, Alert, Card } from 'react-bootstrap';
import styles from './App.module.css';
function App() {
 const [showAlert, setShowAlert] = useState(true);
 return (
  <div className={`container ${styles.appContainer}`}>
   {showAlert && (
    <Alert variant="info" onClose={() => setShowAlert(false)} dismissible>
     Welcome to React + Bootstrap!
    </Alert>
```

```
        )}

        <Card className="mt-4">
          <Card.Body>
            <Card.Title>Styled React App</Card.Title>
            <Card.Text>
              Using Bootstrap and custom CSS modules.
            </Card.Text>
            <Button variant="success">Get Started</Button>
          </Card.Body>
        </Card>
      </div>
    );
  }
  export default App;
```

App.module.css

```
  .appContainer {
    max-width: 800px;
    margin: 0 auto;
    padding: 20px;
  }
```

Summary

- **Inline styles**: Good for dynamic styles.
- **CSS Modules**: Best for scoped styles.
- **Styled Components**: Best for dynamic CSS-in-JS.
- **Bootstrap**: Use react-bootstrap for better React integration.
- **Tailwind CSS**: Best for utility-first styling.

Choose the method that best fits your project needs!

6. Event Handling in React

1. Introduction to Event Handling in React

React events are similar to DOM events but with some key differences:

- **Synthetic Events**: React wraps native browser events for cross-browser compatibility.
- **CamelCase Naming**: onClick instead of onclick.
- **JSX Syntax**: Pass functions instead of strings.

2. Basic Event Handling

Handling Click Events

```
function Button() {
  const handleClick = () => {
    alert('Button clicked!');
  };
  return <button onClick={handleClick}>Click Me</button>;
}
```

Inline Arrow Function

```
<button onClick={() => alert('Inline click!')}>Click Me</button>
```

3. Passing Arguments to Event Handlers

Using Arrow Functions

```
function UserList() {
  const users = ['Alice', 'Bob', 'Charlie'];
  const handleDelete = (userId) => {
    console.log('Deleting user:', userId);
  };
  return (
    <ul>
      {users.map((user, index) => (
        <li key={index}>
          {user}
          <button onClick={() => handleDelete(index)}>Delete</button>
```

```
      </li>
    ))}
  </ul>
 );
}
```

Using .bind() **(Less Common)**

```
<button onClick={handleDelete.bind(null, index)}>Delete</button>
```

4. Common React Events

Event Type	React Syntax	Description
Click	onClick	Mouse click
Change	onChange	Input change (forms)
Submit	onSubmit	Form submission
Key Events	onKeyDown, onKeyUp	Keyboard interactions
Mouse Events	onMouseEnter, onMouseLeave	Hover effects
Focus/Blur	onFocus, onBlur	Input focus changes

Example: Form Handling

```
function LoginForm() {
 const [email, setEmail] = useState('');
 const [password, setPassword] = useState('');
 const handleSubmit = (e) => {
  e.preventDefault(); // Prevent page reload
  console.log({ email, password });
 };
 return (
  <form onSubmit={handleSubmit}>
```

```
      <input
        type="email"
        value={email}
        onChange={(e) => setEmail(e.target.value)}
        placeholder="Email"
      />
      <input
        type="password"
        value={password}
        onChange={(e) => setPassword(e.target.value)}
        placeholder="Password"
      />
      <button type="submit">Login</button>
    </form>
  );
}
```

5. Event Object (SyntheticEvent)

React passes a **SyntheticEvent** object to handlers with useful methods:

- e.preventDefault() – Prevent default browser behavior.
- e.stopPropagation() – Stop event bubbling.
- e.target – The DOM element that triggered the event.

Example: Stopping Propagation

```
function ParentComponent() {
  const handleParentClick = () => {
    console.log('Parent clicked');
  };
  return (
    <div onClick={handleParentClick}>
      <button onClick={(e) => {
```

```
        e.stopPropagation();
        console.log('Button clicked');
      }}>
        Click Me
      </button>
    </div>
  );
}
```

6. Custom Events (Child-to-Parent Communication)

Pass event handlers as props to child components.

Example: Child Triggering Parent Function

```
function Parent() {
  const handleChildEvent = (message) => {
    alert(`Child says: ${message}`);
  };
  return <Child onAction={handleChildEvent} />;
}
function Child({ onAction }) {
  return <button onClick={() => onAction('Hello!')}>Trigger Parent</button>;
}
```

7. Best Practices

1. **Avoid Inline Arrow Functions in Render** (Can cause re-renders)

   ```
   // ✕ Avoid (creates new function on each render)
   <button onClick={() => handleClick(id)}>Delete</button>
   // ✓ Better (memoize handler)
   const handleClick = useCallback((id) => { ... }, []);
   <button onClick={() => handleClick(id)}>Delete</button>
   ```

2. **Use** useCallback **for Expensive Handlers**

   ```
   const handleClick = useCallback(() => { ... }, [dependencies]);
   ```

3. **Destructure** e.target **for Cleaner Code**

```
const handleChange = (e) => {
 const { name, value } = e.target;
 setFormData(prev => ({ ...prev, [name]: value }));
};
```

8. Advanced: Event Delegation

For dynamic lists, attach a single event listener to the parent:

```
function UserList() {
 const handleClick = (e) => {
  if (e.target.tagName === 'BUTTON') {
   const userId = e.target.dataset.id;
   console.log('Deleting:', userId);
  }
 };
 return (
  <ul onClick={handleClick}>
   {users.map(user => (
    <li key={user.id}>
     {user.name}
     <button data-id={user.id}>Delete</button>
    </li>
   ))}
  </ul>
 );
}
```

7. Component Composition & Event Propagation

1. Component Composition in React

Component composition is a pattern where components are combined to create more complex UIs. Instead of inheriting behavior, React favors composition over inheritance.

Key Concepts

1. **Containment (Children Prop)**
    - **Components can accept arbitrary children via props.children.**
    - **Useful for wrappers like modals, cards, or layout components.**

Example: Card Component

```
function Card({ children }) {
 return <div className="card">{children}</div>;
}
// Usage
<Card>
 <h2>Title</h2>
 <p>Content goes here</p>
</Card>
```

2. **Specialized Components**
    - **Break down components into smaller reusable pieces.**
    - **Example: A Dialog component that uses Header, Body, and Footer.**

```
function Dialog({ title, content, onClose }) {
 return (
  <div className="dialog">
   <DialogHeader title={title} onClose={onClose} />
   <DialogBody>{content}</DialogBody>
   <DialogFooter>
    <button onClick={onClose}>Close</button>
   </DialogFooter>
  </div>
 );
}
```

3. **Render Props**
    - **A component receives a function as a prop to dynamically render content.**
    - **Useful for reusable logic (e.g., fetching data).**

Example: Mouse Tracker

```
function MouseTracker({ render }) {
  const [position, setPosition] = useState({ x: 0, y: 0 });
  const handleMouseMove = (e) => {
    setPosition({ x: e.clientX, y: e.clientY });
  };
  return <div onMouseMove={handleMouseMove}>{render(position)}</div>;
}
// Usage
<MouseTracker render={({ x, y }) => (
  <p>Mouse position: {x}, {y}</p>
)} />
```

4. **Higher-Order Components (HOCs)**
    - **A function that takes a component and returns an enhanced version.**
    - **Example: withAuth(ProfilePage) to add authentication checks.**

```
function withAuth(Component) {
  return function AuthenticatedComponent(props) {
    const isLoggedIn = checkAuth();
    return isLoggedIn ? <Component {...props} /> : <LoginPage />;
  };
}
```

2. Event Propagation in React

React events bubble up the component tree (similar to DOM events). Understanding propagation is crucial for managing complex interactions.

How Event Bubbling Works

- **Events triggered on a child component propagate ("bubble up") to parent components.**
- **Example: Clicking a button inside a div triggers handlers on both the button and the div.**

Stopping Propagation

- **Use e.stopPropagation() to prevent an event from reaching parent components.**

Example: Preventing a Modal Close

```
function Modal({ onClose }) {
 const handleContentClick = (e) => {
  e.stopPropagation(); // Prevents the modal backdrop from closing
 };
 return (
  <div className="modal-backdrop" onClick={onClose}>
   <div className="modal-content" onClick={handleContentClick}>
    <h2>Modal Title</h2>
    <button onClick={onClose}>Close</button>
   </div>
  </div>
 );
}
```

Event Capturing (Trickling Down)

- **React supports the capture phase (before bubbling) via onClickCapture.**
- **Rarely used, but helpful for advanced cases.**

Example: Logging Events in Capture Phase

```
<div onClickCapture={() => console.log("Parent captured click")}>
 <button onClick={() => console.log("Button clicked")}>
  Click Me
 </button>
</div>
```

Best Practices for Event Handling

1. **Avoid Excessive stopPropagation**
    - **Overusing it can break expected behavior (e.g., nested menus).**
2. **Use Event Delegation**

- Attach a single event listener to a parent for dynamic lists.

```
function List({ items }) {
 const handleClick = (e) => {
  if (e.target.tagName === 'LI') {
   console.log("Item clicked:", e.target.dataset.id);
  }
 };
 return (
  <ul onClick={handleClick}>
   {items.map(item => (
    <li key={item.id} data-id={item.id}>{item.name}</li>
   ))}
  </ul>
 );
}
```

3. Prevent Default Behavior When Needed
   - Use e.preventDefault() for form submissions or anchor tags.

```
<a href="#" onClick={(e) => {
 e.preventDefault();
 console.log("Link clicked but no navigation");
}}>
 Click Me
</a>
```

3. Combining Composition & Event Propagation

Example: A Reusable Dropdown Component

```
function Dropdown({ children, title }) {
 const [isOpen, setIsOpen] = useState(false);
```

```
  const toggleDropdown = (e) => {
    e.stopPropagation(); // Prevents document-wide click handlers from closing it
    setIsOpen(!isOpen);
  };
  useEffect(() => {
    const closeDropdown = () => setIsOpen(false);
    document.addEventListener("click", closeDropdown);
    return () => document.removeEventListener("click", closeDropdown);
  }, []);

  return (
    <div className="dropdown" onClick={toggleDropdown}>
      <button>{title}</button>
      {isOpen && <div className="dropdown-menu">{children}</div>}
    </div>
  );
}

// Usage
<Dropdown title="Menu">
  <button onClick={() => alert("Option 1")}>Option 1</button>
  <button onClick={() => alert("Option 2")}>Option 2</button>
</Dropdown>
```

Key Takeaways

1. **Composition makes components reusable and modular.**
2. **Event propagation can be controlled with stopPropagation().**
3. **Capture phase (onClickCapture) is useful for advanced cases.**
4. **Always clean up event listeners in useEffect.**

Summary Table

Concept	Description	Example
Containment	**Nest components via children**	**<Card><Content /></Card>**
Render Props	**Pass JSX via a function prop**	**<MouseTracker render={position => ...} />**
Event Bubbling	**Events propagate up the tree**	**<div onClick><button /> **
stopPropagation	**Halts event bubbling**	**e.stopPropagation()**
Event Delegation	**Single listener for dynamic lists**	**<ul onClick={handleItemClick}>**

Final Thoughts

- **Prefer composition over inheritance for flexible UIs.**
- **Understand event flow to avoid unintended side effects.**
- **Use patterns like render props/HOCs for cross-cutting concerns.**

By mastering these concepts, you can build scalable and maintainable React applications!

8. Working with Forms in React

Forms are essential for user input in web applications. React handles forms differently than traditional HTML because it manages form data using **state**, making forms **controlled components**. Below is a detailed explanation of how forms work in React.

1. Controlled vs. Uncontrolled Components

Controlled Components

- Form elements (like <input>, <textarea>, <select>) are controlled by React state.
- The **value** of the input is tied to a state variable.
- Changes are handled via onChange event.

Example:

```
import { useState } from 'react';
function Form() {
 const [name, setName] = useState('');
 const handleChange = (e) => {
  setName(e.target.value); // Updates state on every keystroke
 };
 return (
  <form>
   <input
    type="text"
    value={name}
    onChange={handleChange}
    placeholder="Enter your name"
   />
   <p>Hello, {name}!</p>
  </form>
 );
}
```

Key Points:

- value={name} binds the input to the state.
- onChange updates state when the user types.

Uncontrolled Components

- Form data is handled by the DOM itself (like traditional HTML forms).
- Use ref to access input values when needed.

Example:

```
import { useRef } from 'react';
function UncontrolledForm() {
 const inputRef = useRef();
```

```
  const handleSubmit = (e) => {
    e.preventDefault();
    alert(`Submitted: ${inputRef.current.value}`);
  };
  return (
    <form onSubmit={handleSubmit}>
      <input type="text" ref={inputRef} />
      <button type="submit">Submit</button>
    </form>
  );
}
```

Key Points:

- No state management for input.
- Values are accessed using ref on submission.

---

2. Handling Form Submission

- Use onSubmit on the <form> element.
- Prevent default page reload with e.preventDefault().

Example:

```
function LoginForm() {
  const [email, setEmail] = useState('');
  const [password, setPassword] = useState('');
  const handleSubmit = (e) => {
    e.preventDefault();
    console.log('Submitted:', { email, password });
  };
  return (
    <form onSubmit={handleSubmit}>
      <input
```

```
        type="email"
        value={email}
        onChange={(e) => setEmail(e.target.value)}
        placeholder="Email"
      />
      <input
        type="password"
        value={password}
        onChange={(e) => setPassword(e.target.value)}
        placeholder="Password"
      />
      <button type="submit">Login</button>
    </form>
  );
}
```

3. Handling Multiple Inputs

Instead of writing separate state handlers for each input, use a single state object and dynamic property names.

Example:

```
function MultiInputForm() {
  const [formData, setFormData] = useState({
    username: '',
    email: '',
    password: ''
  });
  const handleChange = (e) => {
    const { name, value } = e.target;
    setFormData({ ...formData, [name]: value });
  };
```

```
const handleSubmit = (e) => {
  e.preventDefault();
  console.log('Submitted:', formData);
};
return (
  <form onSubmit={handleSubmit}>
    <input
      type="text"
      name="username"
      value={formData.username}
      onChange={handleChange}
      placeholder="Username"
    />
    <input
      type="email"
      name="email"
      value={formData.email}
      onChange={handleChange}
      placeholder="Email"
    />
    <input
      type="password"
      name="password"
      value={formData.password}
      onChange={handleChange}
      placeholder="Password"
    />
    <button type="submit">Submit</button>
  </form>
```

```
  );
 }
```

Key Points:

- name attribute matches the state key.
- Spread operator (...) preserves unchanged fields.

4. Form Validation

- Check input values before submission.
- Display error messages if validation fails.

Example:

```
function ValidatedForm() {
 const [email, setEmail] = useState('');
 const [error, setError] = useState('');
 const handleSubmit = (e) => {
  e.preventDefault();
  if (!email.includes('@')) {
   setError('Invalid email!');
   return;
  }
  setError('');
  console.log('Submitted:', email);
 };
 return (
  <form onSubmit={handleSubmit}>
   <input
    type="email"
    value={email}
    onChange={(e) => setEmail(e.target.value)}
    placeholder="Email"
   />
```

```
      {error && <p style={{ color: 'red' }}>{error}</p>}
      <button type="submit">Submit</button>
    </form>
  );
}
```

---

5. Working with <select>**,** <textarea>**, and Checkboxes**

Select Dropdown

```
const [country, setCountry] = useState('');
<select value={country} onChange={(e) => setCountry(e.target.value)}>
  <option value="">Select Country</option>
  <option value="us">USA</option>
  <option value="uk">UK</option>
</select>
```

Textarea

```
const [bio, setBio] = useState('');
<textarea
  value={bio}
  onChange={(e) => setBio(e.target.value)}
  placeholder="Tell us about yourself"
/>
```

Checkbox

```
const [isChecked, setIsChecked] = useState(false);
<input
  type="checkbox"
  checked={isChecked}
  onChange={(e) => setIsChecked(e.target.checked)}
/>
```

---

6. Form Libraries (Advanced)

For complex forms, consider libraries like:

- **Formik** (Simplifies form state management)
- **React Hook Form** (Optimized for performance)
- **Yup** (For schema validation)

Example with React Hook Form:

```
import { useForm } from 'react-hook-form';
function HookForm() {
 const { register, handleSubmit } = useForm();
 const onSubmit = (data) => {
  console.log(data);
 };
 return (
  <form onSubmit={handleSubmit(onSubmit)}>
   <input {...register('username')} placeholder="Username" />
   <input {...register('email')} placeholder="Email" />
   <button type="submit">Submit</button>
  </form>
 );
}
```